

HPC Viva Questions

Question 1: What is Parallel Computing?

Answer:

Parallel Computing is a type of computation where multiple tasks are executed simultaneously using multiple processors, cores, or threads to solve a problem faster.

Instead of one processor doing all work sequentially, the work is divided into smaller sub-tasks and executed together in parallel.

Reasoning (Beginner Understanding):

Imagine one person solving 100 math problems alone. It takes a long time.

Now imagine 10 people solving 10 problems each simultaneously. The total work finishes much faster.

This is the basic idea of Parallel Computing.

In HPC:

- CPU cores,
- GPU cores,
- threads,
- and distributed systems

work together to improve speed and performance.

Question 2: What is the difference between Sequential Computing and

Parallel Computing?

Answer:

Sequential Computing	Parallel Computing
One task executes at a time	Multiple tasks execute together
Uses single processor/core mainly	Uses multiple cores/processors
Slower for large computations	Faster for large computations
Simpler to implement	More complex synchronization
Lower performance	Higher performance

Reasoning (Beginner Understanding):

In sequential execution:

```
Task1 → Task2 → Task3
```

text

In parallel execution:

```
Task1
Task2 → Executed Together
Task3
```

text

Parallel computing reduces execution time by distributing work.
GPU computing and CUDA are based on this idea.

Question 3: What is OpenMP?

Answer:

OpenMP (Open Multi-Processing) is an API used for parallel programming in C, C++, and Fortran using shared-memory architecture.

It uses compiler directives like:

```
#pragma omp parallel
```

cpp

to create multiple threads and execute code in parallel.

Reasoning (Beginner Understanding):

Normally, a program runs using one thread.

OpenMP allows us to create multiple threads automatically.

Example:

```
#pragma omp parallel for
```

cpp

This divides loop iterations among multiple threads.

If a loop has 100 iterations and 4 threads:

- Thread 1 handles 0–24
- Thread 2 handles 25–49
- Thread 3 handles 50–74
- Thread 4 handles 75–99

This improves performance on multi-core CPUs.

Question 4: What is CUDA?

Answer:

CUDA (Compute Unified Device Architecture) is a parallel computing platform developed by NVIDIA that allows programmers to use GPU power for general-purpose computing.

CUDA enables execution of thousands of threads simultaneously on the GPU.

Reasoning (Beginner Understanding):

Normally, GPUs are used for graphics.

CUDA allows programmers to use GPU cores for:

- matrix operations,
- AI/ML,
- scientific computing,
- simulations,
- and parallel algorithms.

GPU contains thousands of lightweight cores.

So operations like matrix multiplication become much faster compared to CPU execution.

Question 5: What is the difference between CPU and GPU?

Answer:

CPU	GPU
Few powerful cores	Thousands of lightweight cores
Optimized for sequential tasks	Optimized for parallel tasks
Better for control operations	Better for repetitive arithmetic
Low throughput	High throughput
Complex architecture	Massive parallel architecture

Reasoning (Beginner Understanding):

CPU is like a highly intelligent manager handling complex decisions.

GPU is like thousands of workers performing the same operation simultaneously.

Example:

- CPU is better for operating systems and logic-heavy tasks.
- GPU is better for matrix multiplication and AI training.

That is why CUDA uses GPU computation.

Question 6: What is a Thread in Parallel Computing?

Answer:

A thread is the smallest unit of execution inside a process.

Multiple threads can execute different parts of a program simultaneously.

In CUDA:

- thousands of threads run together.

In OpenMP:

- threads are created automatically using compiler directives.
-

Reasoning (Beginner Understanding):

Suppose a task has multiple independent parts.

Instead of one worker doing everything:

- multiple workers perform different parts simultaneously.

Each worker is like a thread.

Threads improve speed by sharing workload.

Question 7: What is a Race Condition?

Answer:

A race condition occurs when multiple threads access and modify the same shared variable simultaneously without proper synchronization.

This can produce unpredictable or incorrect results.

Reasoning (Beginner Understanding):

Suppose two threads execute:

```
sum = sum + 1;
```

cpp

at the same time.

Both threads may read the old value together.

As a result:

- one update may be lost.

This creates incorrect output.

Race conditions are prevented using:

- critical sections,
 - atomic operations,
 - locks,
 - or reduction clauses.
-

Question 8: What is Synchronization in Parallel Computing?

Answer:

Synchronization is the process of coordinating multiple threads to ensure correct execution order and safe access to shared resources.

It prevents data inconsistency and race conditions.

Reasoning (Beginner Understanding):

Imagine multiple people writing on the same whiteboard simultaneously.

Without coordination:

- writing overlaps,
- information gets corrupted.

Synchronization ensures:

- only safe and controlled access occurs.

In OpenMP:

- `critical`
- `barrier`
- `atomic`
- `taskwait`

are synchronization mechanisms.

Question 9: What is BFS (Breadth First Search)?

Answer:

Breadth First Search (BFS) is a graph traversal algorithm that explores nodes level-by-level using a queue.

It visits all neighboring nodes before moving to the next depth level.

Reasoning (Beginner Understanding):

Suppose you start from one person in a social network.

BFS first visits:

- all direct friends, then:
- friends of friends, then:
- next levels.

It works level-wise.

BFS uses:

Queue (FIFO)

text

Because of level-wise traversal, BFS is easier to parallelize.

Question 10: What is DFS (Depth First Search)?

Answer:

Depth First Search (DFS) is a graph traversal algorithm that explores as deep as possible along one branch before backtracking.

DFS uses recursion or stack data structure.

Reasoning (Beginner Understanding):

Imagine exploring a maze.

DFS keeps moving forward until:

- no path remains.

Then it backtracks and tries another path.

DFS works depth-wise.

It uses:

Stack (LIFO)text

DFS is harder to parallelize because traversal depends on previous exploration path.

Question 11: What is the difference between BFS and DFS?

Answer:

BFS	DFS
Breadth-wise traversal	Depth-wise traversal
Uses Queue	Uses Stack/Recursion
Finds shortest path in unweighted graph	Does not guarantee shortest path
Easier to parallelize	Harder to parallelize
Visits neighbors first	Goes deep first

Reasoning (Beginner Understanding):

BFS explores level-by-level.

Example:

```
Level 1 → Level 2 → Level 3
```

text

DFS explores one complete path before returning.

Example:

```
Path A → Deepest Node → Backtrack
```

text

BFS is better when shortest path is needed.

DFS is useful when complete exploration is needed.

Question 12: Why is BFS easier to parallelize than DFS?

Answer:

BFS is easier to parallelize because all nodes at the same level can be processed simultaneously.

DFS is difficult to parallelize because traversal depends on the current path and recursive exploration order.

Reasoning (Beginner Understanding):

In BFS:

- nodes at one level are independent.
- multiple threads can process them together.

Example:

All can run in parallel.

In DFS:

- next node depends on current node.
- recursion creates dependencies.

This makes synchronization difficult.

Question 13: What is Parallel Reduction?

Answer:

Parallel Reduction is a technique where multiple threads combine values together using operations like:

- sum,
- minimum,
- maximum,
- multiplication.

The final result becomes a single output value.

Reasoning (Beginner Understanding):

Suppose we want sum of:

1 2 3 4 5 6 7 8

text

Sequentially:

- one thread adds everything.

In parallel reduction:

- Thread 1 adds first half
- Thread 2 adds second half

Then partial sums are combined.

This reduces execution time significantly.

Question 14: What is the purpose of reduction clause in OpenMP?

Answer:

The reduction clause allows safe parallel computation on shared variables by creating private copies for each thread and combining them automatically at the end.

Example:

```
#pragma omp parallel for reduction(+:sum)
```

cpp

Reasoning (Beginner Understanding):

Without reduction:

- multiple threads update same variable simultaneously.
- race condition occurs.

With reduction:

- each thread maintains local copy.
- final values are merged safely.

This improves correctness and performance.

Question 15: What is a Kernel in CUDA?

Answer:

A kernel is a special function executed on the GPU by many threads in parallel.

CUDA kernels are launched using:

```
kernel<<<blocks, threads>>>();
```

cpp

Reasoning (Beginner Understanding):

In normal C++:

- functions execute on CPU.

In CUDA:

- kernels execute on GPU.

When kernel launches:

- thousands of GPU threads execute the same function simultaneously on different data.

Example:

- vector addition,
 - matrix multiplication,
 - image processing.
-

Question 16: What are Blocks and Grids in CUDA?

Answer:

CUDA organizes threads hierarchically:

- Threads form a Block.
- Blocks form a Grid.

This structure helps manage large-scale parallel execution.

Reasoning (Beginner Understanding):

Suppose:

- one block contains 256 threads.
- 100 blocks exist.

Total threads:

```
256 × 100 = 25600 threads
```

text

Grid → collection of blocks.

Block → collection of threads.

This organization allows GPU to schedule work efficiently.

Question 17: What is thread indexing in CUDA?

Answer:

Thread indexing is used to identify the unique thread executing a computation.

Common indexing formula:

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

cpp

Reasoning (Beginner Understanding):

Every thread must know:

- which data element it should process.

Example:

- Thread 0 processes A[0]
- Thread 1 processes A[1]
- Thread 2 processes A[2]

Thread indexing provides unique IDs to threads.

Without indexing:

- all threads may process same data.
-

Question 18: What is cudaMalloc()?

Answer:

`cudaMalloc()` is a CUDA function used to allocate memory on GPU device memory.

Syntax:

```
cudaMalloc((void**)&ptr, size);
```

cpp

Reasoning (Beginner Understanding):

CPU memory and GPU memory are separate.

Before GPU computation:

- memory must be allocated on GPU.

`cudaMalloc()` works similar to:

```
malloc()
```

cpp

but allocation occurs on GPU memory instead of CPU RAM.

Question 19: What is cudaMemcpy()?

Answer:

`cudaMemcpy()` is used to transfer data between CPU memory and GPU memory.

Syntax:

```
cudaMemcpy(destination, source, size, direction);
```

cpp

Reasoning (Beginner Understanding):

GPU cannot directly access normal CPU variables.

So data transfer is required.

Two common transfers:

```
Host → Device  
Device → Host
```

text

Example:

- input vectors copied to GPU.
 - result copied back to CPU.
-

Question 20: What is cudaDeviceSynchronize()?

Answer:

`cudaDeviceSynchronize()` pauses CPU execution until all GPU kernel executions are completed.

It ensures correct synchronization between CPU and GPU.

Reasoning (Beginner Understanding):

GPU execution is asynchronous.

That means:

- CPU may continue execution before GPU finishes.

If result is copied too early:

- incorrect output may occur.

`cudaDeviceSynchronize()` ensures:

- GPU work completes fully before proceeding.
-

Question 21: Why do we use GPU for Parallel Computing?

Answer:

GPU is used for parallel computing because it contains thousands of lightweight cores capable of executing many operations simultaneously.

GPUs are highly efficient for:

- matrix operations,
- vector processing,
- AI/ML,
- simulations,
- image processing,
- and scientific computing.

Reasoning (Beginner Understanding):

CPU contains few powerful cores.

GPU contains thousands of smaller cores.

Suppose we must add millions of numbers.

CPU:

- processes fewer operations at a time.

GPU:

- divides operations among thousands of threads.

This massively improves throughput and speed.

Question 22: What is SIMT Architecture?

Answer:

SIMT stands for Single Instruction Multiple Threads.

It is the execution model used in CUDA GPUs where multiple threads execute the same instruction on different data simultaneously.

Reasoning (Beginner Understanding):

Suppose we want to add two vectors.

Every thread performs:

```
C[i] = A[i] + B[i]
```

text

The instruction is same for all threads.

But:

- Thread 1 handles element 1
- Thread 2 handles element 2
- Thread 3 handles element 3

Same instruction + different data = SIMT.

Question 23: What is a Warp in CUDA?

Answer:

A warp is a group of 32 CUDA threads that execute instructions together inside the GPU.

Warp is the basic scheduling unit in CUDA architecture.

Reasoning (Beginner Understanding):

GPU does not schedule threads individually.

Instead:

- 32 threads are grouped together.

This group is called a warp.

If threads inside a warp follow different execution paths:

- performance decreases.

This problem is called warp divergence.

Question 24: What is Warp Divergence?

Answer:

Warp divergence occurs when threads inside the same warp execute different branches of conditional statements.

This reduces GPU efficiency because threads cannot execute fully in parallel.

Reasoning (Beginner Understanding):

Suppose:

```
if(i % 2 == 0)                                     cpp
```

Some threads execute IF block. Other threads execute ELSE block.

GPU then executes branches separately.

This reduces parallel efficiency because warp execution becomes partially sequential.

Question 25: What is the difference between Host and Device in CUDA?

Answer:

Host	Device
CPU side	GPU side
Executes normal C/C++ code	Executes CUDA kernels
Uses system RAM	Uses GPU memory
Controls execution	Performs parallel computation

Reasoning (Beginner Understanding):

In CUDA:

- CPU is called Host.
- GPU is called Device.

Workflow:

1. Host prepares data.
2. Data transferred to Device.
3. Device performs parallel computation.
4. Result copied back to Host.

This cooperation enables GPU acceleration.

Question 26: What is Shared Memory in CUDA?

Answer:

Shared memory is a fast on-chip memory shared among threads within the same block.

It is much faster than global memory.

Reasoning (Beginner Understanding):

Accessing global GPU memory is slower.

Shared memory acts like:

- a small high-speed cache for threads.

Threads in same block can:

- store intermediate data,
- reuse values,
- reduce global memory accesses.

This improves CUDA performance significantly.

Question 27: What is Global Memory in CUDA?

Answer:

Global memory is the main GPU memory accessible by all CUDA threads.

It has large storage capacity but higher access latency.

Reasoning (Beginner Understanding):

Global memory is like GPU RAM.

Advantages:

- accessible by all blocks and threads.
- large storage capacity.

Disadvantage:

- slower compared to shared memory.

Efficient CUDA programs try to minimize unnecessary global memory access.

Question 28: Why is Matrix Multiplication highly suitable for GPU execution?

Answer:

Matrix multiplication is highly suitable for GPU execution because:

- many independent calculations occur simultaneously,
- each output element can be computed independently,
- GPU cores can process multiple elements in parallel.

Reasoning (Beginner Understanding):

In matrix multiplication:

```
C[row][col]
```

text

Each element calculation is independent.

So:

- Thread 1 computes $C[0][0]$
- Thread 2 computes $C[0][1]$
- Thread 3 computes $C[1][0]$

Thousands of threads work together simultaneously.

This matches GPU architecture perfectly.

Question 29: Why is Vector Addition less beneficial on GPU compared to Matrix Multiplication?

Answer:

Vector addition gives smaller GPU speedup because:

- computation is simple,
 - memory transfer overhead dominates,
 - arithmetic intensity is low.
-

Reasoning (Beginner Understanding):

Vector addition performs only:

```
A[i] + B[i]
```

```
text
```

This operation is extremely simple.

Time spent transferring data between CPU and GPU may become larger than actual computation time.

Matrix multiplication performs many arithmetic operations.

Therefore:

- GPU utilization becomes much higher.
- speedup becomes larger.

Question 30: What is the purpose of bounds checking in CUDA kernels?

Answer:

Bounds checking prevents threads from accessing invalid memory locations when total threads exceed actual data size.

Example:

```
if(i < N)
```

```
cpp
```

Reasoning (Beginner Understanding):

Suppose:

- vector size = 1000
- launched threads = 1024

Extra threads:

- should not access memory beyond array size.

Without bounds checking:

- segmentation faults,
- invalid memory access,
- or incorrect outputs may occur.

Bounds checking ensures safe execution.

Question 31: What is Speedup in Parallel Computing?

Answer:

Speedup measures how much faster a parallel program executes compared to a sequential program.

Formula:

$$\text{Speedup} = \text{Sequential Time} / \text{Parallel Time}$$

text

Reasoning (Beginner Understanding):

Suppose:

- Sequential execution time = 10 seconds
- Parallel execution time = 2 seconds

Then:

$$\text{Speedup} = 10 / 2 = 5$$

text

This means:

- parallel execution is 5 times faster.

Higher speedup indicates better parallel performance.

Question 32: What is Efficiency in Parallel Computing?

Answer:

Efficiency measures how effectively processor resources are utilized in parallel execution.

Formula:

$$\text{Efficiency} = \text{Speedup} / \text{Number of Processors}$$

text

Reasoning (Beginner Understanding):

Suppose:

- Speedup = 8
- Processors = 10

Then:

$$\text{Efficiency} = 8 / 10 = 0.8$$

text

Meaning:

- system uses 80% of processing capability efficiently.

Perfect efficiency is difficult because:

- communication overhead,
 - synchronization,
 - and idle threads reduce performance.
-

Question 33: What is Amdahl's Law?

Answer:

Amdahl's Law states that the maximum improvement from parallelization is limited by the sequential portion of a program.

Formula:

$$\text{Speedup} = 1 / ((1 - P) + P/N)$$

text

Where:

- P = parallel portion
- N = number of processors

Reasoning (Beginner Understanding):

Suppose:

- 90% program is parallel
- 10% remains sequential

Even with infinite processors:

- sequential part still limits total speedup.

So complete infinite speedup is impossible.

This law explains practical limitations of parallel systems.

Question 34: What is Scalability in Parallel Computing?

Answer:

Scalability is the ability of a parallel system to maintain performance improvement when processors or workload increase.

Reasoning (Beginner Understanding):

Suppose we increase:

- number of threads,
- GPU cores,
- processors.

A scalable system should:

- continue improving performance.

Poor scalability occurs when:

- synchronization overhead increases,
 - communication becomes expensive,
 - threads remain idle.
-

Question 35: What is Load Balancing?

Answer:

Load balancing is the process of distributing workload evenly among processors or threads.

It prevents some processors from being overloaded while others remain idle.

Reasoning (Beginner Understanding):

Suppose:

- 4 workers exist.

If:

- Worker 1 gets 90 tasks,
- others get 10 tasks,

then system becomes inefficient.

Balanced workload ensures:

- all processors stay active,
 - better performance is achieved.
-

Question 36: What is Communication Overhead?

Answer:

Communication overhead is the extra time spent transferring data or synchronizing between processors instead of performing actual computation.

Reasoning (Beginner Understanding):

In parallel systems:

- processors often exchange data.

This communication consumes time.

Example:

- CPU ↔ GPU memory transfer
- thread synchronization
- network communication in clusters

Too much communication reduces performance benefits.

Question 37: What is Data Parallelism?

Answer:

Data parallelism is a parallel computing technique where the same operation is performed simultaneously on different pieces of data.

Reasoning (Beginner Understanding):

Suppose we have:

```
A = [1,2,3,4]
B = [5,6,7,8]
```

text

Different threads compute:

```
C[0] = A[0] + B[0]
C[1] = A[1] + B[1]
```

text

Same instruction. Different data.

CUDA vector addition uses data parallelism.

Question 38: What is Task Parallelism?

Answer:

Task parallelism occurs when different threads execute different tasks simultaneously.

Reasoning (Beginner Understanding):

Suppose one program contains:

- sorting,
- searching,
- file reading.

Different threads can perform different operations simultaneously.

Unlike data parallelism:

- operations are different.

Task parallelism is common in multi-threaded CPU applications.

Question 39: Why is synchronization important in parallel programs?

Answer:

Synchronization is important because multiple threads may access shared data simultaneously.

Without synchronization:

- race conditions,
 - inconsistent outputs,
 - and corrupted data may occur.
-

Reasoning (Beginner Understanding):

Suppose two threads update same variable:

```
sum += value;
```

cpp

at the same time.

Without synchronization:

- one update may overwrite another.

Synchronization ensures:

- safe and correct execution order.
-

Question 40: Why do we use `omp_get_wtime()` in OpenMP?

Answer:

`omp_get_wtime()` is used to measure execution time in parallel programs.

It provides high-resolution wall-clock timing.

Reasoning (Beginner Understanding):

To compare:

- sequential performance,
- parallel performance,

we need accurate timing.

Example:

```
double start = omp_get_wtime();
```

cpp

Execution begins.

```
double end = omp_get_wtime();
```

cpp

Execution ends.

Difference gives total execution time.

This helps analyze speedup and efficiency.

Question 41: What is Matrix Multiplication in Parallel Computing?

Answer:

Matrix multiplication is a computation where elements of two matrices are combined to produce a new output matrix.

In parallel computing:

- multiple output elements are computed simultaneously using threads.
-

Reasoning (Beginner Understanding):

Suppose:

```
C = A × B
```

text

Each output element:

```
C[row][col]
```

text

is independent.

Therefore:

- one thread can compute one output element.

This makes matrix multiplication highly suitable for GPU execution.

Question 42: Why is Merge Sort better than Bubble Sort for Parallel Computing?

Answer:

Merge Sort is better for parallel computing because it uses divide-and-conquer strategy, allowing independent subproblems to execute simultaneously.

Bubble Sort contains many dependencies between adjacent elements.

Reasoning (Beginner Understanding):

Merge Sort:

- divides array into halves.
- both halves can be sorted simultaneously.

Bubble Sort:

- repeatedly compares neighboring elements.
- operations depend on previous swaps.

Therefore:

- Merge Sort scales better in parallel systems.
-

Question 43: What is the purpose of `#pragma omp parallel for`?

Answer:

`#pragma omp parallel for` automatically divides loop iterations among multiple threads for parallel execution.

Reasoning (Beginner Understanding):

Suppose loop has:

```
for(int i=0;i<100;i++)
```

cpp

OpenMP divides iterations among available threads.

Example:

- Thread 1 handles 0–24
- Thread 2 handles 25–49
- Thread 3 handles 50–74

- Thread 4 handles 75–99

This reduces execution time significantly.

Question 44: What is `#pragma omp critical`?

Answer:

`#pragma omp critical` ensures that only one thread executes a critical section at a time. It protects shared resources from race conditions.

Reasoning (Beginner Understanding):

Suppose multiple threads print output simultaneously.

Without critical section:

- outputs may overlap.

Example:

```
#pragma omp critical
{
    sum += value;
}
```

cpp

Only one thread updates `sum` at a time.

This ensures safe execution.

Question 45: What is the difference between Shared Memory and Distributed Memory?

Answer:

Shared Memory	Distributed Memory
All processors share same memory	Each processor has private memory
Communication through shared variables	Communication through messages
Easier programming	More complex programming
Example: OpenMP	Example: MPI

Reasoning (Beginner Understanding):

In shared memory:

- all threads access same RAM.

In distributed memory:

- processors have separate memory spaces.
- data transfer happens using communication protocols.

OpenMP uses shared memory. MPI uses distributed memory.

Question 46: What is MPI?

Answer:

MPI (Message Passing Interface) is a parallel programming model used for communication between multiple processes in distributed memory systems.

Reasoning (Beginner Understanding):

In cluster computing:

- different computers/processes cannot directly access same memory.

So they communicate using messages.

MPI provides functions for:

- sending data,
- receiving data,
- synchronization,
- broadcasting.

MPI is widely used in supercomputers.

Question 47: What is the difference between OpenMP and CUDA?

Answer:

OpenMP	CUDA
CPU-based parallelism	GPU-based parallelism
Uses multi-core CPU threads	Uses thousands of GPU threads
Shared-memory programming	GPU accelerator programming
Easier implementation	More hardware-specific
Best for moderate parallelism	Best for massive parallelism

Reasoning (Beginner Understanding):

OpenMP:

- improves CPU performance using threads.

CUDA:

- uses GPU hardware for massive parallel execution.

Example:

- OpenMP → parallel loops on CPU.
 - CUDA → matrix multiplication on GPU.
-

Question 48: What is HPC (High Performance Computing)?

Answer:

High Performance Computing (HPC) refers to the use of powerful parallel computing systems to solve large and complex computational problems efficiently.

Reasoning (Beginner Understanding):

Normal computers cannot efficiently solve:

- weather simulations,
- AI training,
- scientific research,
- molecular simulations,
- large-scale analytics.

HPC systems combine:

- CPUs,
- GPUs,
- clusters,
- supercomputers,
- and parallel algorithms.

This provides extremely high computational power.

Question 49: How is Parallel Computing useful in AI and Machine Learning?

Answer:

Parallel computing accelerates AI and Machine Learning by executing large matrix operations, tensor computations, and neural network training simultaneously.

Reasoning (Beginner Understanding):

AI training involves:

- millions of mathematical operations.

GPUs execute these operations in parallel.

Examples:

- matrix multiplication,
- convolution,
- gradient calculations,
- tensor operations.

Without parallel computing:

- deep learning training would become extremely slow.
-

Question 50: What are the main advantages of Parallel Computing?

Answer:

Main advantages of Parallel Computing are:

1. Faster execution
 2. Better resource utilization
 3. Ability to solve large problems
 4. Scalability
 5. Improved performance for scientific and AI applications
-

Reasoning (Beginner Understanding):

Instead of one processor doing all work:

- multiple processors cooperate together.

Benefits:

- reduced execution time,
- efficient hardware usage,
- support for large-scale computations.

Modern technologies like:

- AI,
- simulations,
- cloud computing,
- scientific research,

heavily depend on parallel computing.
